

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

- 1 – **Python**: será impresso -4, pois é o que resultado -2 elevado a 2, e depois 512.
- 2 – **Java**: será impresso 339, pois 3 é o resultado da primeira soma, depois a string "3" é concatenada, seguida do segundo resultado de soma que é 9.
- 3 – **JavaScript**: o resultado é 3 e 52, pois na operação de subtração, a conta com string e inteiro sera permitida, já na segunda operação, a soma irá concatenar a string com o inteiro.
- 4 – **C**: imprimirá 0 pois não há nenhum valor definido para %d
- 5 – **Python**: imprime a string definida e o resultado booleano da comparação, que é true.
- 6 – **JavaScript**: true, pois 0 e "" são vazios, false pois as strings são distintas, e true pois o 0 é considerado igual mesmo com um sendo inteiro e o outro string.

Respondendo as perguntas da página 5 do slide, que envolvem os assuntos trabalhados nas páginas 6, 7, 8 e 9: **Precedência e associatividade; Ordem dos operandos e efeitos colaterais; Operadores sobrecarregados; Conversões de tipos.**

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

a + b * c com a=3, b=4, c=5

da esquerda para a direita
(3 + 4) * 5 = 35

da direita para a esquerda
3 + (4 * 5) = 23

com precedência (* antes de +)
3 + 4 * 5 = 23

APL: sempre da direita para a esquerda (p. 304)
3 x 4 + 5 = 27

- **1. Regras de precedência:** Definem qual operador tem prioridade e deve ser calculado primeiro quando possuem níveis diferentes (ex: multiplicação antes de adição).
- **2. Regras de associatividade:** Definem se operadores de mesma prioridade em uma linha são avaliados da esquerda para a direita (comum em somas) ou da direita para a esquerda (comum em potências).
- **3. Ordem de avaliação dos operandos:** Define se o valor da esquerda ou o da direita de um operador é processado primeiro pelo processador.
- **4. Restrições a efeitos colaterais:** Decidem se uma função dentro de uma expressão pode alterar o valor de outras variáveis na memória enquanto roda. Algumas linguagens proíbem isso para evitar bugs.
- **5. Sobrecarga de operadores:** Define se o programador pode mudar o significado de um símbolo. Linguagens como C++ e Python permitem (ex: usar + para somar matrizes); linguagens como Java proíbem.
- **6. Mistura de tipos:** Define as regras de conversão automática quando tipos diferentes se encontram na mesma conta (ex: somar um inteiro com um número decimal).

Conversões de tipos: alargamento, estreitamento e coerção.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;           // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com a digitado por engano no lugar de `c`, o Java converte e compila.

- **Alargamento:** Converter um tipo menor para um tipo maior. Não há perda de dados (ex: transformar int em float).
- **Estreitamento:** Converter um tipo maior para um tipo menor. Há risco de perder dados ou precisão (ex: transformar 3.99 em 3 ao converter de float para int).
- **Coerção:** É qualquer conversão automática feita pela própria linguagem de programação durante uma operação (ex: em `5 + 2.5`, a linguagem transforma o 5 em 5.0 por conta própria).

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;  
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...  
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)  
    x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Respondendo a pergunta do título, de acordo com o livro do Sebesta, podem ser necessárias apenas duas: sequência, a execução padrão do código, onde as sentenças são executadas uma após a outra, na ordem em que foram escritas (ex: definir a e depois calcular b), e seleção capacidade de escolher caminhos alternativos de execução no código com base em uma condição lógica (ex: desvios condicionais como if/else). A iteração é para repetir um bloco de código enquanto uma certa condição ainda for verdadeira.

E a pergunta no final do slide para cada estrutura: por que há tantas outras estruturas de controle. O for substitui um bloco while que precisa de inicialização, condição e incremento. O switch substitui o que seria uma sequência com diversos if-else deixando a leitura mais limpa. O do while garante que o bloco de código rode pelo menos uma vez, e o foreach elimina a necessidade de gerenciar de forma manual os índices, como com i++ por exemplo. Então por isso há tantas outras estruturas de controle.

Seleção de dois caminhos e Seleção múltipla.

Seleção de dois caminhos permite que o fluxo escolha exatamente duas alternativas para seguir com base no resultado de uma expressão.

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
        int nota = 6;  
        if (nota >= 6) {  
            System.out.printf("Aprovado");  
        } else {  
            System.out.printf("Reprovado");  
        }  
    }  
}
```

Aprovado

Seleção múltipla permite escolher 3 ou mais caminhos de execução possíveis, geralmente com switch case para evitar longas if else.

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        int opcao = 2;
        switch (opcao) {
            case 1: iniciar(); break;
            case 2: configurar(); break;
            default: sair();
        }
    }
}
```

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

1 - JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

2 - Python

```
def desconto(preco, d=None):
    d = d or 10 # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

1 – JavaScript: a saída é 0012, a regra que causa o problema é a iteração ocorrer sobre as chaves do array e não sobre números, e para corrigir basta iterar sobre um inteiro

Versão errada:

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

Total: 0012

Versão corrigida:

```
const precos = [10, 20, 30];
let total = 0;
for (const p of precos) // Corrigido
    total += p;
console.log("Total: " + total);
```

Total: 60

2 – Python: a saída é 180, a regra que causa problema é que o número 0 é avaliado como se fosse false, e a correção deve ser feita garantindo que 0 seja aceito como um valor válido.

Versão errada:

```
def desconto(preco, d=None):  
    d = d or 10 # padrão: 10%  
    return preco * (100 - d) / 100  
print(desconto(200, 0))
```

180.0

Versão corrigida:

```
def desconto(preco, d=None):  
    if d is None: # Corrigido para verificar explicitamente a ausência de valor  
        d = 10  
    return preco * (100 - d) / 100  
  
print(desconto(200, 0)) # Saída correta: 200.0
```

200.0